

Devoir surveillé

Durée 1h30

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
Toute réponse doit être justifiée.*

Exercice 1 (Écho es-tu là ? (les signaux) - 7 pts)

Nous souhaitons réaliser une application qui se met en attente de signaux. Dès qu'elle en reçoit un, elle renvoie immédiatement le même signal au processus qui lui a envoyé.

1. Que se passe-t-il si l'application reçoit trop de signaux ? Pourquoi ?
2. Comment un processus peut-il récupérer le PID du processus qui lui a envoyé un signal ?
3. Pourquoi l'utilisation de `sigaction` n'est pas appropriée dans cette application ?
4. À l'aide de `sigwaitinfo`, donnez le code de l'application (sans les includes et sans la gestion d'erreur).
5. Que se passe-t-il si cette application reçoit le signal `SIGKILL` ?

Nous souhaitons maintenant pouvoir envoyer en même temps que le signal, un message de type `char*`.

6. Est-ce possible ? Si oui, expliquez comment le mettre en œuvre. Sinon, expliquez pourquoi.

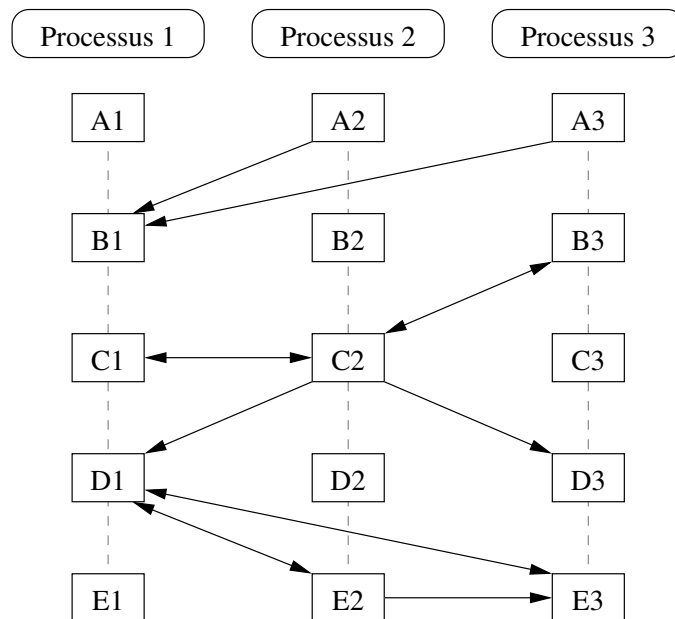
Exercice 2 (Disque dur virtuel - 9 pts)

Nous souhaitons réaliser une application de disque dur virtuel dans un fichier binaire. Un programme permet de copier ou de récupérer des fichiers "réels" dans/depuis le disque virtuel. Pour cela, nous utilisons une structure (nommée `index`) située au début du fichier permettant de représenter les entrées (les fichiers stockés dans le disque dur virtuel). Chaque entrée est identifiée par un nom de 16 caractères maximum et une position dans le fichier.

1. Pourquoi est-il préférable que l'index possède un nombre d'entrées fixe ?
2. Si le nombre d'entrées est trop grand, nous souhaitons pouvoir ajouter un autre index dans le disque dur virtuel. Expliquez comment procéder.
3. Comment représenter les entrées vides ?
4. Proposez une structure en C pour l'index (vous pouvez utiliser d'autres structures si nécessaire).
5. Écrivez la fonction `int creer_disque(char *nom_disque)` qui permet de créer un nouveau disque virtuel dont le nom est passé en paramètre et qui retourne le descripteur de fichier correspondant.
6. Donnez le code de la fonction `off_t rechercher(int fd, char *nom_fichier)` permettant de retourner la position d'un fichier dans le disque virtuel (ou 0 si non trouvé). Le paramètre `fd` est le descripteur de fichier correspondant au disque virtuel (on ne connaît pas la position actuelle du curseur) et `nom_fichier` est le nom du fichier recherché.
7. Expliquez comment ajouter un nouveau fichier dans le disque virtuel (détaillez bien les appels système nécessaires).

Exercice 3 (Sémaphores de Dijkstra - 4 pts)

Soit le diagramme de précédence suivant, représentant 3 processus contenant chacun 5 blocs s'exécutant les uns après les autres :



1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

2°) Proposez une solution pour réduire le nombre de sémaphores.

3°) Expliquez pourquoi la solution proposée pour résoudre les contraintes entre C1, C2 et B3 n'est pas satisfaisante avec les sémaphores de Dijkstra.

Annexes : quelques en-têtes de fonctions C

```

void      _exit(int code);
unsigned int alarm(unsigned int nb_secondes);
int       atexit(void (*procedure)(void));
void      clearerr(FILE *flux);
int       close(int fd);
int       closedir(DIR *repertoire);
int       creat(const char *chemin, mode_t mode);
int       dup(int ancien_fd);
int       dup2(int ancien_fd, nouveau_fd);
int       execve(const char *fichier, char *const argv[], char *const envp[]);
void      exit(int statut);
int       fcntl(int fd, int commande, ...);
int       fclose(FILE *flux);
FILE      *fdopen(int fd, const char *mode);
int       feof(FILE *flux);
int       ferror(FILE *flux);
int       fflush(FILE *flux);
int       fileno(FILE *flux);
FILE      *fopen(const char *chemin, const char *mode);
pid_t     fork();
size_t    fread(void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
FILE      *freopen(const char *chemin, const char *mode, FILE *flux);
int       fseek(FILE *flux, long offset, int point_depart);
int       fstat(int fd, struct stat *tampon);
int       ftell(FILE *flux);
size_t    fwrite(const void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
pid_t     getpid();
pid_t     getppid();
int       kill(pid_t pid, int numero_signal);
off_t     lseek(int fd, off_t offset, int point_depart);
int       lstat(const char *chemin, struct stat *tampon);
void      *memcpy(void *destination, const void *source, size_t taille);
void      *memset(void *tampon, int caractere, size_t nombre_elements);
int       mkfifo(const char *nomFichier, mode_t mode);
int       open(const char *chemin, int options);
int       open(const char *chemin, int options, mode_t mode);
DIR        *opendir(const char *chemin);
int       pause(void);
int       pipe(int tube[2]);
ssize_t    read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void      rewind(FILE *flux);
void      rewinddir(DIR *repertoire);
void      (*signal(int numero_signal, void (*handler)(int)))(int)
int       sigaction(int numero_signal, const struct sigaction *action,
                    struct sigaction *ancienne_action);
int       sigaddset(sigset_t *ensemble, int numero_signal);
int       sigdelset(sigset_t *ensemble, int numero_signal);
int       sigemptyset(sigset_t *ensemble);
int       sigfillset(sigset_t *ensemble);
int       sigismember(sigset_t *ensemble, int numero_signal);
int       sigpending(sigset_t *ensemble);
int       sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int       sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int       sigsuspend(const sigset_t *ensemble);
int       sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int       sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                      const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
int       stat(const char *chemin, struct stat *tampon);

```

```

time_t      time(time_t *temps);
int         truncate(const char *chemin, off_t offset);
int         ftruncate(int fd, off_t offset);
int         truncate64(const char *chemin, off64_t offset);
int         ftruncate64(int fd, off64_t offset);
int         unlink(const char *nomFichier);
pid_t       wait(int *statut);
pid_t       waitpid(pid_t pid, int *statut, int options);
ssize_t     write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

```

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH,
S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_depart (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP,
S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED

```

Annexes : macros

Sur `statut` de `wait`, `waitpid` : `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`
 Sur le champ `st_mode` de `struct stat` : `S_ISREG`, `S_ISDIR`, `S_ISFIFO`

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};

union sigval_t {
    int sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```